DUY TAN UNIVERSITY

INTERNATIONAL SCHOOL

INDIVIDUAL ASSIGNMENT

SUBJECT: ASIA COMMUNITY WITH IT

SUBMIT BY

NAME	NGUYEN VAN DUY MANH
STUDENT CODE	29211100024
SUBJECT CODE	DTE-CS 231 BIS

Da Nang, March 29, 2026

TABLE OF CONTENTS

1. AI Agent Concept Exploration 2

   1.1. Core Concept & Paradigm Shift 2

   1.2. Defining Characteristics 2

   1.3. Agent Categories & Growing Importance 2

   1.4. Limitations & Human-AI Collaboration 3

2. In-Depth Case Study Analysis 3

   2.1. Case 1: GitHub Copilot Workspace (Software Development) 3

   2.2. Case 2: Jira AI (Project Management) 4

3. AI Agent Application Proposal: AutoFP Estimator 4

   3.1. Problem & Motivation 4

   3.2. Business Value 5

   3.3. Core Definition 5

   3.4. Key Functionalities 5

   3.5. System Workflow 5

   3.6. System Architecture 6

   3.7. Risks & Trade-offs 6

   3.8. Future Development 6

4. Conclusion 7

5. Declaration of AI Usage 7

1. AI Agent Concept Exploration

1.1. Core Concept & Paradigm Shift

- Modern software systems have evolved from deterministic programs toward more adaptive, goal-oriented computing. Traditional software follows predefined logic and executes instructions exactly as written, relying on fixed control flows designed by developers.
- In contrast, an AI agent focuses on achieving objectives rather than simply executing instructions. Developers define the desired outcome, while the agent determines how to reach it based on the situation. This shift allows systems to respond more effectively to dynamic or unexpected conditions.
- From my experience in software projects, traditional systems often fail when inputs or environments change unexpectedly. AI agents address this limitation by adapting their behavior instead of stopping or producing errors.

1.2. Defining Characteristics

- AI agents are distinguished by four key operational characteristics:
    - **Autonomy**: Operate independently, executing background processes and triggering workflows without continuous human input.
    - **Perception**: Interpret complex environments by processing Application Programming Interfaces **(APIs)**, logs, and user interfaces.
    - **Decision-Making:** Use probabilistic reasoning to evaluate multiple possible actions and select the most optimal path.
    - **Adaptability:** Adjust strategies dynamically when encountering changes, such as User Interface **(UI)** modifications or system updates.
- These capabilities allow AI agents to move beyond static automation into intelligent orchestration.

1.3. Agent Categories & Growing Importance

- AI agents are generally classified into three main types:
    - **Reactive Agents:** Execute predefined actions based on specific immediate inputs.
    - **Goal-Based Agents:** Plan, evaluate, and optimize actions over time to achieve complex objectives.
    - **Learning Agents:** Improve their performance over time by learning from data and user feedback.

- Their importance is rapidly increasing due to the exponential growth of data and the complexity of modern cloud systems. Since human teams cannot continuously monitor large-scale distributed architectures, AI agents have become essential for scalable, real-time decision-making.
- To illustrate how AI agents function in practice compared to traditional software, consider these everyday examples:
  - **E-commerce Price Comparison:** A traditional script only scrapes prices from fixed URLs and breaks if the website changes. An AI agent, however, can autonomously navigate different storefronts, adapt to unexpected UI changes, and intelligently match product variations to find the best deals without human intervention.
  - **Smart Home Thermostats:** Instead of following a rigid timer set by the user, an AI agent actively learns a family's daily routine and adjusts the temperature dynamically based on real-time weather conditions and occupancy.

1.4. Limitations & Human-AI Collaboration

- Despite their capabilities, AI agents have critical limitations:
  - **Hallucination:** Generating incorrect or fabricated outputs under uncertainty.
  - **Context Constraints:** Limited ability to process large amounts of information simultaneously.
- Due to these risks, AI agents are not fully reliable in high-stakes environments such as banking or healthcare without strict human validation and regulatory oversight.
- The most effective deployment model is Human-in-the-Loop **(HITL)**, where AI handles data processing while humans provide validation and strategic decisions, ensuring both efficiency and reliability.

2. In-Depth Case Study Analysis

2.1. Case 1: GitHub Copilot Workspace (Software Development)

- GitHub Copilot Workspace functions as a goal-based agent that analyzes repository-level context to generate multi-file implementation plans.
  - Strengths:
    - Rapid scaffolding and prototyping
    - Efficient generation of Create, Read, Update, Delete **(CRUD)** operations and unit tests
  - Limitations:

        - Performs poorly with non-standard or legacy architectures
        - Limited by context window constraints
    - Insight: It significantly accelerates development but behaves like a junior developer, requiring senior-level review for correctness and security.

➔ This demonstrates how AI agents can process large, unstructured inputs and transform them into structured outputs, highlighting the importance of context-aware understanding at a system-wide level.

2.2. Case 2: Jira AI (Project Management)

- Jira AI acts as an analytical agent, leveraging Natural Language Processing **(NLP)** to summarize discussions and estimate workloads.
    - Strengths:
        - Reduces onboarding time
        - Summarizes large volumes of textual data
    - Limitations:
        - Inaccurate for novel or unfamiliar technologies
        - Dependent on historical data quality
    - Insight: Highly effective for administrative automation but lacks deep technical reasoning.

➔ This highlights the importance of extracting meaningful insights from large volumes of unstructured textual data, which is a critical capability for AI agents operating in complex documentation environments.

3. AI Agent Application Proposal: AutoFP Estimator

3.1. Problem & Motivation

- Estimating software size using Function Points **(FP)** is a time-consuming and error-prone process. Analysts must manually interpret large Software Requirements Specification **(SRS)** documents, leading to inefficiency and inconsistency.
- The AutoFP Estimator aims to automate this process using AI, transforming manual estimation into a fast, objective workflow.
- Based on my experience working with requirement documents, I found that even small misunderstandings in interpretation can lead to significant estimation errors. This makes the process not only time-consuming but also unreliable.

3.2. Business Value

- The system targets:
  - IT outsourcing companies
  - Freelance teams
  - Consulting firms
- By reducing estimation time from days to minutes, it lowers operational costs and improves pricing accuracy. The platform follows a Business-to-Business (B2B) Software-as-a-Service (SaaS) model, generating revenue through subscription tiers.
- For example, reducing estimation time by 50–70% can significantly lower labor costs, especially for outsourcing companies handling multiple projects.

3.3. Core Definition

- Name: AutoFP Estimator
- Users: Architects, Project Managers, Quality Assurance Leads
- Objective: Automatically calculate Unadjusted Function Points **(UFP)** from unstructured SRS documents
- Unlike traditional Function Point tools that rely heavily on manual input, the AutoFP Estimator introduces automation and adaptive learning, significantly reducing human effort while maintaining consistency.

3.4. Key Functionalities

- Document Parsing using NLP
- Boundary Identification
- Transaction Classification (External Inputs - **EI**, External Outputs - **EO**, External Inquiries - **EQ**)
- Data Structure Detection (Logical Internal Files - **ILF**, External Interface Files - **EIF**)
- UFP Calculation (International Function Point Users Group - **IFPUG** standard)
- Ambiguity Detection for unclear requirements

3.5. System Workflow

1. User uploads SRS document
2. System parses and analyzes content

3. Functional elements are classified
4. UFP is calculated automatically
5. Results are visualized and exported

- This pipeline significantly reduces manual effort and increases consistency.
- The system operates as an AI agent by continuously refining its classification results based on human feedback. When users correct misclassified elements, the agent updates its internal reasoning patterns, improving accuracy over time. This feedback loop allows the system to adapt to different project domains and writing styles.
- In a typical scenario, a project manager may need to analyze a 40–60 page SRS document. Instead of spending several days manually identifying functional components, the system can generate a structured estimation within minutes.

3.6. System Architecture

- The system follows a modular AI architecture:
  - **Input Layer:** Document ingestion and preprocessing
  - **NLP Layer:** Semantic parsing and entity extraction
  - **Classification Engine:** Maps entities to FP categories
  - **Calculation Module:** Applies IFPUG formulas
  - **Validation Layer:** Human review and correction
  - **Output Layer:** Dashboard and API integration
- This architecture ensures scalability, maintainability, and enterprise integration.

3.7. Risks & Trade-offs

- Loss of Human Intuition
- NLP Misclassification Errors
- Dependency on Text-Based Input
- However, continuous model training can improve accuracy over time.

3.8. Future Development

- Future enhancements may include:
  - Fine-tuned domain-specific Large Language Models **(LLMs)**

  - Multimodal processing (images, diagrams)
  - Continuous learning from feedback
  - Integration with DevOps pipelines
- These improvements would transform the system into a fully adaptive intelligent estimation platform.

4. Conclusion

- The effectiveness of AI agents depends not on technical complexity but on selecting the right problem domain. By targeting real inefficiencies, AI systems can deliver significant business value.
- The AutoFP Estimator demonstrates how automating software estimation can reduce costs, improve accuracy, and streamline workflows. While challenges such as NLP errors and data variability remain, the benefits clearly outweigh the limitations.
- This evolution signals a broader shift where software systems are no longer merely tools, but collaborative decision-making partners in complex engineering environments.
- From my perspective, the key challenge is not only building the AI agent, but ensuring that it can be trusted in real-world scenarios where accuracy directly impacts business decisions.

5. Declaration of AI Usage

- In completing this assignment, the core ideas, domain selection, and system architecture for the AutoFP Estimator were developed through my own original analysis and discussions with my project team. I utilized AI tools (ChatGPT/Gemini) primarily as a refinement assistant. Specifically, I drafted the initial concepts and structures myself, then used AI to improve the English phrasing, check grammar, and ensure a professional tone. All case study evaluations, business models, and final conclusions reflect my personal critical thinking and understanding of the subject matter.